

AeroTwin

Real-Time Digital Twin for Predictive Aircraft Engine Health Monitoring

✈️ AEROSPACE MRO

🔧 EDGE AI + ML

📊 DIGITAL TWIN

TEAM FELONS

PROJECT DETAILS

Team FELONS

Prakash Tiwari (Lead) Backend · Architecture · ML

Harnoor Kaur Frontend · UI/UX · Demo

Prince Sagwal Simulation Engine · Database

Piyush Kumar ML Pipeline · Documentation

Event: Tata Technologies InnoVent-27

Theme: AI at the Edge

Track: AI at the Edge Solutions for Aerospace

Sub-Category: Edge AI for Digital Twin-Enabled MRO

AI Categories: Edge AI + Predictive AI / Machine Learning

Engine Type: Commercial Turbofan (C-MAPSS calibrated)

Dataset: NASA C-MAPSS (FD001–FD004)

Status: Requirement Analysis & Planning

WHAT ARE WE BUILDING — AEROTWIN IN PLAIN LANGUAGE

AeroTwin is a real-time software dashboard that watches over a commercial aircraft turbofan engine's health — continuously, automatically, and intelligently. It ingests sensor data (temperature, vibration, RPM) either from the NASA C-MAPSS dataset or from a built-in live simulation engine, processes it through a fatigue calculation algorithm and a machine learning model, and displays the engine's health score, predicted Remaining Useful Life (RUL), and maintenance recommendations on a live interactive dashboard — all updating in real time without any human intervention.

Think of it like a doctor doing a continuous ECG on a patient instead of checking their pulse once a day. The dashboard is what the doctor sees on the monitor — always live, always current, always ready to fire an alert the moment something looks wrong.

THE CORE IDEA IN ONE LINE

Instead of waiting for a scheduled maintenance inspection to discover a problem with an aircraft engine — which could be days or weeks after the problem started — AeroTwin detects it within seconds of it appearing, and tells you exactly what to do about it.

For teammates reading this: we are not connecting to a real jet engine. We are building a software system that takes engine sensor data — either from a real NASA research dataset (CSV mode) or from our own simulation engine (Live ML mode) — and makes it look and behave exactly like a live monitoring system would in a real airline MRO environment. The intelligence, the architecture, and the dashboard are all real. Only the physical sensor is replaced by software data.

ENGINE SCOPE — WHICH ENGINE ARE WE MONITORING?

Why We Had to Pick One Specific Engine Type

All aircraft engines are different. A commercial passenger jet engine (turbofan), a cargo plane engine, a fighter jet engine, a helicopter engine (turboshaft), and a propeller aircraft engine (turboprop) all operate on fundamentally different principles, at different temperatures, RPMs, vibration levels, and degradation rates. You cannot build one monitoring system and claim it works for all of them — that would be scientifically incorrect.

AeroTwin is specifically scoped to: Commercial Turbofan Engines — the engines that power passenger aircraft like the Boeing 737 (CFM56 engine), Airbus A320 (CFM56 or IAE V2500), Boeing 777 (GE90), and Airbus A350 (Trent XWB). This is the engine family that NASA's C-MAPSS simulation dataset models, and it represents the largest and most commercially urgent segment of the aviation MRO market.

Engine Type	Example Aircraft	AeroTwin Scope?	Why / Why Not
Commercial Turbofan	Boeing 737, Airbus A320/A380, Boeing 777	✔️ YES — Primary scope	Modelled by NASA C-MAPSS. Largest MRO market. Most publicly available research data.
Military Turbofan (with afterburner)	F-16 (F110), F-35 (F135), Tejas (F404)	❌ Not in scope	Classified data, different operating envelope, afterburner dynamics not in

			C-MAPSS.
Turboprop	ATR 72, Dash 8, C-130 Hercules	✗ Not in scope	Propeller coupling dynamics fundamentally different. No matching public dataset.
Turboshaft	Black Hawk helicopter (T700)	✗ Not in scope	Helicopter rotor coupling changes degradation physics completely.
High-Bypass Cargo Turbofan	Boeing 747F (CF6), 777F (GE90)	📄 Partial overlap	Similar to passenger turbofan — our model partially applies but cargo duty cycles differ.

➔ OFFICIAL SCOPE STATEMENT

AeroTwin monitors commercial turbofan engine health, calibrated against the NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) turbofan dataset. It tracks three component health streams: High-Pressure Compressor (HPC), Fan/Turbine Blade, and Bearing — mapped from C-MAPSS fault modes FD003 and FD004.

THE NASA C-MAPSS DATASET — OUR DATA FOUNDATION

► What Is NASA C-MAPSS?

C-MAPSS stands for Commercial Modular Aero-Propulsion System Simulation. It is a high-fidelity software simulator of a commercial turbofan engine developed by NASA's Prognostics Center of Excellence (PCoE) at Ames Research Center. The dataset generated from C-MAPSS is the most widely used benchmark for aircraft engine predictive maintenance research globally — cited in over 1,000 academic papers and used as the standard training dataset by researchers at NASA, MIT, ETH Zurich, and hundreds of other institutions.

Critically: C-MAPSS data is itself simulated — generated by a software model of an engine, not by sensors on a real engine. This is exactly the same approach AeroTwin uses, and it is the standard methodology in aerospace predictive maintenance research.

► Dataset Download Links

📄 OFFICIAL DOWNLOAD LINKS

Primary Source (NASA Direct):
<https://phm-datasets.s3.amazonaws.com/NASA/6.+Turbofan+Engine+Degradation+Simulation+Data+Set.zip>

NASA C-MAPSS 2 (Enhanced — 5.3M samples, real flight conditions):
<https://phm-datasets.s3.amazonaws.com/NASA/17.+Turbofan+Engine+Degradation+Simulation+Data+Set+2.zip>

Kaggle Mirror (easier download, same data):

<https://www.kaggle.com/datasets/behrad3d/nasa-cmaps>

NASA Official Data Portal:

<https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data>

NASA PCoE Repository (all datasets):

<https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>

► Dataset Contents — What You Get

The C-MAPSS dataset contains four sub-datasets (FD001–FD004) of increasing complexity. We use FD003 and FD004 as they include TWO fault modes (HPC degradation + Fan degradation) matching our three monitored components:

Dataset	Train Engines	Test Engines	Operating Conditions	Fault Modes
FD001	100	100	1 (Sea Level)	1 — HPC Degradation only
FD002	260	259	6 (Multiple altitudes)	1 — HPC Degradation only
FD003 ✓	100	100	1 (Sea Level)	2 — HPC + Fan Degradation
FD004 ✓	248	249	6 (Multiple altitudes)	2 — HPC + Fan Degradation

► The 21 Sensor Channels — Mapped to AeroTwin Components

Each row in C-MAPSS represents one flight cycle and contains: engine_id, cycle_number, 3 operational settings, and 21 sensor readings. Here is how we map C-MAPSS sensors to AeroTwin's three monitored components:

C-MAPSS Sensor	Physical Meaning	AeroTwin Component	Role in Fatigue Formula
s2 — Total temperature at HPC outlet	Temperature after high-pressure compressor stage	🔧 Compressor Health	Primary temperature factor for compressor fatigue score
s3 — Total temperature at HPT outlet	Temperature exiting high-pressure turbine	➔ Turbine/Fan Blade Health	Primary temperature factor for blade fatigue score
s4 — Total pressure at HPT outlet	Pressure drop across high-pressure turbine	➔ Turbine/Fan Blade Health	Pressure-based stress indicator for blade fatigue
s7 — Total pressure at HPC outlet	Compressor outlet pressure	🔧 Compressor Health	Compressor loading indicator
s8 — Physical fan speed (Nf)	Actual fan rotation speed (RPM)	⚙️ Bearing Health	Primary RPM factor for bearing fatigue score
s9 — Physical core speed (Nc)	Core shaft rotation speed (RPM)	➔ Turbine/Fan Blade Health	Core RPM driving blade centrifugal stress
s11 — Static pressure at	Internal compressor	🔧 Compressor Health	Supporting compressor

HPC outlet	pressure		stress indicator
s12 — Fuel flow ratio (Wf)	Fuel flow to combustion chamber	All components	Thermal load proxy for all component fatigue
s13 — Corrected fan speed	Fan speed corrected for altitude/temp	⚙️Bearing Health	Bearing load indicator under varying conditions
s14 — Corrected core speed	Core speed corrected for conditions	➡️Turbine/Fan Blade Health	Blade stress under varying flight phases
s15 — Bypass ratio	Ratio of bypass to core airflow	All components	Engine operating mode indicator
s17 — Bleed enthalpy	Energy extracted from core for cabin/systems	🔧 Compressor Health	Compressor workload indicator
s20 — HPT cool bleed flow	Cooling air fed to high-pressure turbine	➡️Turbine/Fan Blade Health	Blade cooling effectiveness — critical for thermal fatigue
s21 — LPT cool bleed flow	Cooling air fed to low-pressure turbine	⚙️Bearing Health	Secondary thermal load on bearing housing

Sensors s1, s5, s6, s10, s16, s18, s19 show near-zero variance across operating conditions and are dropped during feature engineering (standard practice in C-MAPSS literature).

TWO DATA MODES — CSV REPLAY VS LIVE ML SIMULATION

AeroTwin supports two completely different data input modes, switchable by the user via a toggle on the dashboard. Both modes feed into the exact same processing pipeline — the fatigue engine, ML model, WebSocket stream, and React dashboard are identical regardless of which mode is active. Only the data source changes.

🔧 TOGGLE DESIGN

The dashboard has a prominent radio-button / toggle switch at the top:

●MODE A: CSV Replay — Use NASA C-MAPSS Dataset (Real Research Data)

●MODE B: Live Simulation — Generate Synthetic Engine Data in Real Time

Switching modes resets the simulation and begins streaming from the selected source immediately. Both modes use the same WebSocket channel, same dashboard, same ML model.

► MODE A — CSV Replay (NASA C-MAPSS Dataset)

In this mode, AeroTwin reads rows from the downloaded NASA C-MAPSS CSV files one-by-one, replaying them as if they were arriving live from a real sensor. This is called 'dataset replay' and is the standard validation technique used in aerospace research to test monitoring systems against known data before connecting real hardware.

HOW IT WORKS — Step by Step:

Step	Action	Who Does This / What File
Step 1 — Download	Team downloads NASA C-MAPSS FD003 or FD004 ZIP from NASA S3 (link in Section 3). Extract files: train_FD003.txt, test_FD003.txt, RUL_FD003.txt	
Step 2 — Preprocess	A Python script (preprocess_cmapss.py in our repo) loads the .txt file into pandas, adds column names (21 sensors + op settings), normalises sensor values to 0–1 range, drops near-zero-variance sensors, and saves as cleaned_cmapss_fd003.csv	
Step 3 — Backend reads CSV	When MODE A is selected, Flask backend reads cleaned CSV into memory. A background thread iterates through rows, one row every 2 seconds — each row = one flight cycle of real C-MAPSS sensor data	
Step 4 — Component mapping	Each CSV row's sensor values are mapped to AeroTwin's three components: s3/s4/s20→Turbine Blade, s2/s7/s11/s17→Compressor, s8/s9/s13/s14→Bearing	
Step 5 — Fatigue engine	Mapped sensor values enter the fatigue calculation formula exactly as in MODE B — producing health scores per component	
Step 6 — ML inference	The trained Random Forest model (trained ON the same C-MAPSS data) receives engineered features and outputs RUL prediction per component	
Step 7 — WebSocket emit	Health scores, RUL predictions, and alerts are emitted via Socket.IO to the React dashboard — updating every 2 seconds	
Step 8 — Dashboard renders	Dashboard displays REAL NASA C-MAPSS sensor data driving REAL component health scores. When a judge asks 'is this real data?' — in MODE A, the answer is: yes, from NASA's own turbofan research dataset	

► MODE B — Live ML Simulation (Synthetic Real-Time Engine Data)

In this mode, AeroTwin's own simulation engine generates sensor readings mathematically in real time. This is the mode used for the live hackathon demonstration because it gives us full control — we can speed up time, slow it down, inject anomalies, and reset to baseline on demand. The synthetic data is calibrated against C-MAPSS parameters to be physically realistic.

HOW IT WORKS — Step by Step:

Step	Action	Technical Detail
Step 1 — Simulation engine starts	Flask background thread launches. Flight_hour counter starts at 0. All components initialised at health=100%	
Step 2 — Generate one reading	Every 2 seconds, the engine generates one set of sensor values per component using the degradation formula: $\text{base_value} + (\text{flight_hour} \times \text{degradation_rate}) + \text{random_noise}$. Parameters calibrated from C-MAPSS FD003 normal operating ranges	
Step 3 — Apply degradation curve	Temperature: rises $\sim 0.8^\circ\text{C}/\text{flight-hour}$ (turbine blade), $\sim 0.5^\circ\text{C}/\text{flight-hour}$ (compressor). Vibration: rises exponentially after 100 flight hours (bearing wear accelerates non-linearly). RPM: slight drift (-0.02% per flight-hour, compressor fouling)	
Step 4 — Fatigue calculation	Same formula as MODE A: $\text{fatigue_delta} = (\text{T_factor}^{1.4} \times \text{V_factor}^{1.8} \times \text{RPM_factor}^{1.2}) \times \text{sensitivity}$. Health = 100 – cumulative_fatigue	
Step 5 — ML inference	Same trained Random Forest model. Takes engineered features from synthetic readings. Outputs RUL prediction and confidence	
Step 6 — Anomaly injection	If judge/operator clicks 'Inject Anomaly' button, simulation spikes one sensor by 3–5× normal value. Anomaly z-score detector fires within same tick. CRITICAL alert generated	
Step 7 — WebSocket emit + Dashboard	Same pipeline as MODE A. Dashboard is visually identical. Only the data origin is different	

► Side-by-Side Comparison — MODE A vs MODE B

Dimension	MODE A — CSV Replay	MODE B — Live Simulation
-----------	---------------------	--------------------------

Data source	NASA C-MAPSS CSV (real research dataset)	Python simulation engine (synthetic)
Data credibility	Highest — NASA-published, 1000+ papers	High — calibrated to C-MAPSS parameters
Demo control	Limited — follows fixed dataset sequence	Full — can inject anomalies, reset, speed up
Internet required?	No — CSV loaded locally at startup	No — fully self-contained
Best used for	Judge Q&A ('is this real data?'), ML model training validation	Live hackathon demonstration
Processing pipeline	Identical to Mode B	Identical to Mode A
Dashboard appearance	Identical	Identical
ML model	Same trained model	Same trained model
Anomaly injection	✗Cannot inject (fixed data)	✓On-demand anomaly injection
Time control	✗Fixed playback speed	✓Configurable simulation speed

SYSTEM ARCHITECTURE — COMPLETE TECHNICAL DESIGN

AeroTwin uses a five-layer architecture. The data source (Layer 1) is the only layer that changes between MODE A and MODE B. Layers 2–5 are completely identical regardless of mode. This is the most important architectural principle of the system.

LAYER 1 — DATA SOURCE LAYER	Mode A: NASA C-MAPSS CSV file replayed row-by-row Mode B: Python simulation engine generating synthetic readings Output: Structured JSON {component_id, flight_hour, temperature, vibration, rpm, timestamp}
↕ MODE-AGNOSTIC BOUNDARY	Everything above this line is identical regardless of whether data comes from CSV or simulation. This boundary is AeroTwin's most important design principle.
LAYER 2 — PROCESSING & INTELLIGENCE LAYER	Fatigue Accumulation Engine (custom multi-factor formula) Random Forest ML model (RUL prediction + confidence) Z-score Anomaly Detector (sudden fault detection) Rule-Based Alert Engine (GREEN/AMBER/RED/CRITICAL)
LAYER 3 — COMMUNICATION LAYER	Flask-SocketIO WebSocket server Persistent bidirectional connection to all React clients 3 event types per tick: sensor_update, health_update, alert_update Latency: < 100ms end-to-end

LAYER 4 — APPLICATION & API LAYER

Python Flask REST API (7 endpoints)
 PostgreSQL database (3 tables — see Section 7)
 Mode toggle endpoint (POST /api/mode)
 Historical data retrieval and anomaly injection

LAYER 5 — PRESENTATION LAYER

React.js dashboard (real-time state via WebSocket)
 Three.js 3D engine component visualization
 Recharts live fatigue trend graphs
 Mode toggle radio button (CSV / Live Simulation)
 Alert panel + Historical Log modal

► Full Data Flow — Every Step from Source to Screen

The following table traces one complete data cycle through the entire AeroTwin pipeline:

Step	Where It Happens	What Happens	File/Module
1	Layer 1 (CSV/Sim)	New reading generated or replayed	simulator.py or csv_reader.py
2	Layer 2	Reading mapped to 3 components via sensor_map dict	component_mapper.py
3	Layer 4 — DB	Raw reading persisted to PostgreSQL sensor_readings table	db_writer.py
4	Layer 2	Fatigue formula applied → health_score per component updated	fatigue_engine.py
5	Layer 2	Last 10 readings fetched from PostgreSQL → 12 features engineered	feature_engineer.py
6	Layer 2	Random Forest model runs inference → predicted_rul + confidence	ml_model.py
7	Layer 4 — DB	Health snapshot saved to PostgreSQL health_snapshots table	db_writer.py
8	Layer 2	Alert engine evaluates RUL thresholds → alert payload if needed	alert_engine.py
9	Layer 4 — DB	Any new alert persisted to PostgreSQL maintenance_log table	db_writer.py

10	Layer 3	SocketIO emits 3 events to all connected clients	socketio_server.py
11	Layer 5 — React	useEffect hook receives events → setState updates	Dashboard.jsx
12	Layer 5 — Three.js	Component mesh color interpolates to new health value	EngineModel.jsx
13	Layer 5 — Recharts	New data point appended to chart series (rolling 200pt window)	HealthChart.jsx
14	Layer 5 — Alert Panel	New alert card prepended if alert_update event received	AlertPanel.jsx

PROJECT FOLDER STRUCTURE — WHO BUILDS WHAT

This section tells every teammate exactly which files and folders exist, what each one does, and who is responsible for building it. When you clone the AeroTwin repo, this is the structure you will create:

File / Folder	What It Does	Responsible	Priority
backend/	Root folder for all Python/Flask server code	All	—
backend/app.py	Flask app entry point. Registers all routes, initialises SocketIO, starts background simulation/CSV thread on startup	Prakash Tiwari	 Critical
backend/simulator.py	MODE B: Generates synthetic sensor readings using degradation formula. Runs as Flask background thread. Emits readings every 2 seconds	Prince Sagwal	 Critical
backend/csv_reader.py	MODE A: Reads preprocessed C-MAPSS CSV row by row. Same timing interface as simulator.py. Drop-in replacement	Prince Sagwal	 Critical
backend/component_mapper.py	Maps raw C-MAPSS sensor columns to AeroTwin's 3 components (Turbine/Compressor/Bearing) using the sensor mapping table from Section 3	Prince Sagwal	 High
backend/fatigue_engine.py	Implements the multi-factor fatigue formula. Maintains running cumulative fatigue	Prakash Tiwari	 Critical

	score per component. Returns health_score per tick		
backend/feature_engineer.py	Queries last 10 readings from PostgreSQL per component. Computes 12 engineered ML features (rolling means, slopes, z-scores, etc.)	Piyush Kumar	🔗 High
backend/ml_model.py	Loads trained Random Forest model (aerotwin_model.pkl). Runs inference on engineered features. Returns predicted_rul + confidence	Piyush Kumar	🔗 Critical
backend/alert_engine.py	Evaluates predicted_rul against thresholds. Also reads z-scores for anomaly detection. Returns alert payload if threshold crossed	Prakash Tiwari	🔗 High
backend/socketio_server.py	Defines all Socket.IO events (sensor_update, health_update, alert_update). Handles inject_anomaly, advance_time, reset, mode_switch client events	Prakash Tiwari	🔗 Critical
backend/db.py	PostgreSQL connection setup using psycopg2 or SQLAlchemy. Database URL loaded from .env. Creates tables if not exist on startup	Prince Sagwal	🔗 High
backend/routes.py	Defines all REST API endpoints (GET /api/health, /api/history, /api/rul, /api/alerts, POST /api/anomaly, /api/reset, /api/mode)	Prakash Tiwari	🔗 High
ml/	Root folder for ML training scripts (run once, not part of live server)	Piyush Kumar	—
ml/preprocess_cmapss.py	Downloads/reads C-MAPSS .txt files. Adds column names, drops low-variance sensors, normalises, saves cleaned CSV to data/	Piyush Kumar	🔗 High
ml/generate_synthetic.py	Generates 10,000 synthetic degradation trajectories using simulator.py logic. Labels each point with true RUL. Saves as training_data.csv	Piyush Kumar	🔗 High
ml/train_model.py	Loads training data (either C-MAPSS or synthetic). Engineers features. Trains Random Forest	Piyush Kumar	🔗 Critical

	Regressor. Evaluates MAE/RMSE. Saves aerotwin_model.pkl		
ml/aerotwin_model.pkl	Trained model binary (generated by train_model.py). Committed to repo. Loaded by ml_model.py at Flask startup	Piyush Kumar	 Critical
data/	NASA C-MAPSS raw and cleaned CSV files (not committed — downloaded by setup script)	Prince Sagwal	—
data/download_dataset.py	Script to download NASA C-MAPSS ZIP from NASA S3 URL, extract, and place files in data/. Team runs this once after cloning	Prince Sagwal	 High
data/cleaned_cmapss_fd003.csv	Output of preprocess_cmapss.py. Used by csv_reader.py in MODE A	Piyush Kumar	 High
frontend/	React app root (created with create-react-app or Vite)	Harnoor Kaur	—
frontend/src/components/Dashboard.jsx	Main layout component. Holds all state. Connects to Socket.IO. Routes incoming events to child components. Contains mode toggle radio button	Harnoor Kaur	 Critical
frontend/src/components/EngineModel.jsx	Three.js scene: renders 3D schematic of engine. 3 meshes for Turbine/Compressor/Bearing. Color interpolates GREEN→AMBER→RED on health score update	Harnoor Kaur	 Critical
frontend/src/components/HealthChart.jsx	Recharts LineChart per component. Shows temperature, vibration, fatigue trends over flight hours. Rolling 200-point window	Harnoor Kaur	 High
frontend/src/components/AlertPanel.jsx	Sidebar list of all active and historical alerts. CRITICAL alerts pulse with CSS animation. Pulls history from GET /api/alerts	Harnoor Kaur	 High
frontend/src/components/RULPanel.jsx	Displays predicted_rul and confidence band per component. Updates on every health_update Socket.IO event	Harnoor Kaur	 High
frontend/src/components/HistoryModal.jsx	Modal that opens on 'View History' click. Calls GET /api/history. Renders full	Harnoor Kaur	 Medium

	sensor log for selected component		
<code>frontend/src/components/ModeToggle.jsx</code>	Radio button UI for CSV/Live mode switch. Calls POST /api/mode on change. Shows active mode badge at top of dashboard	Harnoor Kaur	🔗 High
<code>.env</code>	Environment variables: DATABASE_URL (PostgreSQL connection string), FLASK_ENV, SIMULATION_SPEED_SECONDS. Never committed to Git	All	🔗 Critical
<code>.env.example</code>	Template .env with placeholder values. Committed to repo so teammates know what variables to set	Prakash Tiwari	🔗 High
<code>requirements.txt</code>	Python dependencies: flask, flask-socketio, flask-cors, psycpg2-binary, scikit-learn, pandas, numpy, joblib, eventlet	Prakash Tiwari	🔗 High
<code>README.md</code>	Setup instructions, how to download dataset, how to run backend and frontend, how to switch modes, team credits	All (Piyush Kumar leads)	🔗 High
<code>docker-compose.yml</code>	Optional: Runs Flask backend + PostgreSQL in Docker for consistent setup across all teammates' machines	Prakash Tiwari	🔗 Medium

DATABASE SCHEMA — POSTGRESQL (REPLACING SQLITE)

► Why PostgreSQL Instead of SQLite

The earlier version of this document used SQLite. We have upgraded to PostgreSQL for the following reasons:

- ◆ **Concurrent writes:** SQLite locks the entire database file during writes. With WebSocket events firing every 2 seconds per component (6 writes/second total), SQLite would create bottlenecks. PostgreSQL handles concurrent writes without locking.
- ◆ **Production readiness:** PostgreSQL is the industry-standard database for production web applications. Using it in AeroTwin makes the codebase directly deployable to cloud hosting (Railway, Heroku, Render — all offer free PostgreSQL tiers).
- ◆ **Docker compatibility:** PostgreSQL runs cleanly in Docker alongside the Flask backend, making teammate setup identical across Windows/Mac/Linux machines.
- ◆ **Future scalability:** When AeroTwin expands to monitor multiple engines or a full fleet, PostgreSQL's indexing and query optimisation will be essential. SQLite has a 281TB theoretical limit but is not designed for high-frequency concurrent time-series writes at scale.

► Connection Setup

Database URL is stored in .env as:

DATABASE_URL=postgresql://aerotwin_user:password@localhost:5432/aerotwin_db

Flask connects via psycopg2-binary (pip install psycopg2-binary). Tables are auto-created on startup if they do not exist via db.py's init_db() function.

► Table 1 — sensor_readings

Stores every raw sensor reading from both MODE A and MODE B. One row per component per simulation tick.

Column	PostgreSQL Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique ID
session_id	UUID	NOT NULL	Groups all readings from one simulation run
mode	VARCHAR(10)	NOT NULL CHECK (mode IN ('csv','live'))	Which data mode generated this reading
component_id	VARCHAR(30)	NOT NULL	turbine_blade / bearing / compressor
flight_hour	INTEGER	NOT NULL	Simulated flight hour (0–∞)
temperature	NUMERIC(8,3)	NOT NULL	Temperature reading in °C
vibration	NUMERIC(8,4)	NOT NULL	Vibration in mm/s
rpm	NUMERIC(10,2)	NOT NULL	Engine RPM
cmapss_engine_id	INTEGER	NULLABLE	C-MAPSS engine ID (MODE A only, NULL in MODE B)
cmapss_cycle	INTEGER	NULLABLE	C-MAPSS cycle number (MODE A only)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Server timestamp of reading

Index: CREATE INDEX idx_sensor_component_session ON sensor_readings(component_id, session_id, flight_hour);

► Table 2 — health_snapshots

Stores the processed health score and RUL prediction for each component on every tick. Used by the dashboard for historical trend display.

Column	PostgreSQL Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique ID
session_id	UUID	NOT NULL	Links to same session as sensor_readings

component_id	VARCHAR(30)	NOT NULL	Component identifier
flight_hour	INTEGER	NOT NULL	Simulated flight hour
fatigue_score	NUMERIC(6,3)	NOT NULL CHECK (0 <= fatigue_score <= 100)	Cumulative fatigue (0=new, 100=failed)
health_score	NUMERIC(6,3)	NOT NULL CHECK (0 <= health_score <= 100)	100 – fatigue_score
predicted_rul	NUMERIC(8,2)	NOT NULL	ML model RUL estimate in flight hours
confidence	NUMERIC(5,4)	NOT NULL CHECK (0 <= confidence <= 1)	Prediction confidence (0–1)
severity	VARCHAR(10)	NOT NULL CHECK (severity IN ('GREEN','AMBER','RED','CRITICAL'))	Current alert level
created_at	TIMESTAMPTZ	DEFAULT NOW()	Snapshot timestamp

Index: CREATE INDEX idx_health_component_session ON health_snapshots(component_id, session_id, flight_hour);

► Table 3 — maintenance_log

Stores every alert generated by the alert engine. Serves as the tamper-evident audit trail for safety review.

Column	PostgreSQL Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique ID
session_id	UUID	NOT NULL	Session that generated this alert
component_id	VARCHAR(30)	NOT NULL	Component that triggered the alert
severity	VARCHAR(10)	NOT NULL	GREEN / AMBER / RED / CRITICAL
recommended_action	TEXT	NOT NULL	Human-readable maintenance instruction
predicted_rul_at_alert	NUMERIC(8,2)	NOT NULL	RUL value that triggered the alert
anomaly_flag	BOOLEAN	DEFAULT FALSE	TRUE if triggered by z-score anomaly detector
z_score_at_alert	NUMERIC(6,3)	NULLABLE	Z-score value if anomaly_flag is TRUE
mode	VARCHAR(10)	NOT NULL	csv or live — which mode was active

created_at	TIMESTAMPZ	DEFAULT NOW()	Timestamp of alert generation
------------	------------	---------------	-------------------------------

MACHINE LEARNING PIPELINE — DETAILED SPEC

► Training Data — Two Sources Used Together

The Random Forest model is trained on a combined dataset: 70% from processed NASA C-MAPSS FD003+FD004 data (real research benchmark), and 30% from synthetically generated degradation trajectories (adds diversity). This dual-source approach gives the model both real-data credibility and synthetic coverage of rare fault scenarios that are underrepresented in C-MAPSS.

► 12 Engineered Features

Feature ID	Feature Name	Why It Matters
F1	rolling_mean_vibration_10	Average vibration over last 10 ticks — removes noise, captures sustained trend
F2	rolling_std_vibration_10	Std deviation of vibration — high std = unstable bearing
F3	vibration_slope_20	Linear slope of vibration over 20 ticks — rate of bearing wear acceleration
F4	rolling_mean_temp_10	Average temperature — sustained heat = thermal fatigue accumulation
F5	temp_rise_rate	°C gained per flight hour — early warning of thermal degradation
F6	rolling_std_temp_10	Temperature instability — indicates combustion irregularity
F7	rpm_drift	Deviation from baseline RPM — compressor fouling signature
F8	vib_temp_correlation	Pearson correlation of vibration vs temperature — high correlation = mechanical-thermal coupling fault
F9	cumulative_fatigue	Physics-based fatigue score — bridges rule-based and ML approaches
F10	health_score	Current health % — direct state snapshot
F11	flight_hour_normalised	Time elapsed as 0–1 — model learns time-dependent degradation acceleration
F12	max_z_score_10	Max anomaly z-score across all streams in last 10 ticks — sudden fault

early warning

► Model Training Specification

- ◆**Algorithm:** Algorithm: Random Forest Regressor (sklearn.ensemble.RandomForestRegressor)
- ◆**Hyperparameters:** n_estimators=100, max_depth=12, min_samples_split=5, random_state=42. Depth capped to reflect edge hardware constraints.
- ◆**Train/Test Split:** 80% train / 20% validation. Stratified by fault type and operating condition to ensure all C-MAPSS sub-datasets are represented in both splits.
- ◆**Validation Metrics:** MAE ~12–18 flight hours, RMSE ~22–30 flight hours, R² ~0.91–0.94 on validation set.
- ◆**Serialisation:** joblib.dump(model, 'ml/aerotwin_model.pkl') — loaded at Flask startup, held in memory for inference.
- ◆**Explainability:** feature_importances_ array from Random Forest displayed in a dashboard widget — shows judges which sensors drive the RUL prediction.

GITHUB REPOSITORY SETUP GUIDE

This section guides the team through creating and structuring the AeroTwin GitHub repository correctly from day one.

► Repository Creation Settings

Field	Value to Enter
Owner	Tiwari1782 (Prakash's account) — or create a GitHub Organisation 'Team-FELONS'
Owner	Tiwari1782 (Prakash's account) — or create a GitHub Organisation 'Team-FELONS'
Repository Name	AeroTwin
Repository Name	AeroTwin
Description	Real-time Digital Twin for predictive aircraft engine health monitoring InnoVent-27 AI at the Edge Team FELONS
Description	Real-time Digital Twin for predictive aircraft engine health monitoring InnoVent-27 AI at the Edge Team FELONS
Visibility	Public ✔— judges and evaluators need to see it
Visibility	Public ✔— judges and evaluators need to see it
Add README	ON ✔— initialises the repo so you can clone immediately
Add README	ON ✔— initialises the repo so you can clone immediately
Add .gitignore	Python ✔— ignores __pycache__, .env, *.pyc, venv/
Add .gitignore	Python ✔— ignores __pycache__, .env, *.pyc, venv/
Add License	MIT License ✔— standard open-source, appropriate for a

Add License

student hackathon project

MIT License ✓— standard open-source, appropriate for a student hackathon project

► After Creating the Repo — First Steps for Each Teammate

Once Prakash creates the repo, every teammate follows these steps:

- ◆ **Prakash (immediately after creating repo):** Add all 3 teammates as Collaborators: Settings → Collaborators → Add People → add their GitHub usernames. Each gets 'Write' access.
- ◆ **All teammates:** Clone: `git clone https://github.com/Tiwari1782/AeroTwin.git`. Create branch: `git checkout -b feat/<your-name>-<feature>`
- ◆ **Prakash:** Add `.env.example` file with all required environment variables (template). Add `requirements.txt`. Push README with project overview.
- ◆ **Prince Sagwal:** Create `backend/` folder skeleton. Push empty Python files (`app.py`, `simulator.py`, `csv_reader.py`, `db.py`) with docstrings explaining what each will do.
- ◆ **Piyush Kumar:** Create `ml/` folder. Push `preprocess_cmapss.py` skeleton and requirements. Download C-MAPSS dataset and push `data/download_dataset.py` script.
- ◆ **Harnoor Kaur:** Initialise React app: `npx create-react-app frontend`. Push initial component folder structure with empty JSX files for each component.

► GitHub Labels to Create (for Issue Tracking)

Create these labels in your repo (Issues → Labels → New Label) so the team can track work cleanly:

`[backend]` `[frontend]` `[ml-pipeline]` `[database]` `[dataset]` `[bug]` `[documentation]` `[priority-high]`

TEAMMATE RESPONSIBILITIES — WHO BUILDS WHAT

This section is written for your teammates to read. Each person should know exactly what they are responsible for building, in what order, and how their work connects to everyone else's.

Prakash Tiwari — Team Lead · Backend · Architecture · ML Integration

You own the backbone of the entire system.

- ◆ Build `app.py` — the Flask entry point that wires together every module
- ◆ Build `socketio_server.py` — the real-time communication layer that makes the dashboard live
- ◆ Build `fatigue_engine.py` — the core algorithm that converts sensor readings into health scores
- ◆ Build `alert_engine.py` — the rule engine that decides GREEN/AMBER/RED/CRITICAL
- ◆ Build `routes.py` — all REST API endpoints
- ◆ Set up `db.py` — PostgreSQL connection and table creation
- ◆ Integrate `ml_model.py` (handed to you by Piyush once trained) into the Flask request cycle
- ◆ Build the mode-switch endpoint (`POST /api/mode`) that toggles between CSV and Live modes
- ◆ Manage GitHub repo — create, invite teammates, review PRs

- ◆ Write `.env.example` and `requirements.txt`

Prince Sagwal — Simulation Engine · CSV Reader · Database · Data Pipeline

You own how data gets into the system — from both sources.

- ◆ Build `simulator.py` — the MODE B live data generator. It must match the same output JSON schema as `csv_reader.py` exactly, so the pipeline above cannot tell the difference
- ◆ Build `csv_reader.py` — MODE A C-MAPSS replay engine. Reads cleaned CSV row by row at the configured speed
- ◆ Build `component_mapper.py` — maps C-MAPSS's 21 sensor columns to AeroTwin's 3 components using the sensor mapping table in Section 3
- ◆ Write `data/download_dataset.py` — one-command script to download NASA C-MAPSS ZIP, extract, and place files correctly
- ◆ Set up the PostgreSQL database schema (3 tables from Section 7) using `db.py`
- ◆ Write `db_writer.py` — handles all INSERT operations to the 3 tables
- ◆ Test that switching modes (CSV ↔ Live) in the running system works without restart

Piyush Kumar — ML Pipeline · Feature Engineering · Model Training · Documentation

You own the intelligence layer — the ML model is AeroTwin's brain.

- ◆ Write `ml/preprocess_cmapss.py` — clean the raw C-MAPSS `.txt` files into a usable CSV
- ◆ Write `ml/generate_synthetic.py` — create 10,000 synthetic degradation trajectories to supplement C-MAPSS training data
- ◆ Write `ml/feature_engineer.py` (shared with backend/) — compute the 12 engineered features from the last 10 readings
- ◆ Write `ml/train_model.py` — train the Random Forest Regressor on combined C-MAPSS + synthetic data, evaluate, and save `aerotwin_model.pkl`
- ◆ Hand `aerotwin_model.pkl` to Prakash for integration into the Flask backend
- ◆ Build `ml/ml_model.py` — the inference wrapper that loads the pkl and runs predictions in the live server
- ◆ Write and maintain `README.md` — complete setup guide, dataset download instructions, how to switch modes, API reference
- ◆ Write the feature importance chart data pipeline (`ml_model.py` exports `feature_importances_` for dashboard display)

Harnoor Kaur — Frontend · React Dashboard · Three.js · UI/UX · Demo

You own everything the judge sees. The dashboard IS AeroTwin from their perspective.

- ◆ Initialise React app (Vite recommended for speed: `npm create vite@latest frontend`)
- ◆ Build `Dashboard.jsx` — the root component. Connect to Socket.IO. Manage all shared state. Route events to child components
- ◆ Build `ModeToggle.jsx` — the radio button toggle between CSV and Live mode. Calls `POST /api/mode`. Shows active mode badge prominently
- ◆ Build `EngineModel.jsx` — Three.js scene with 3 labeled component meshes. Health-based color interpolation GREEN→AMBER→RED. Hover tooltips
- ◆ Build `HealthChart.jsx` — Recharts LineChart per component. Temperature, vibration, fatigue trends. Rolling 200-point window
- ◆ Build `RULPanel.jsx` — shows predicted_rul and confidence band per component. Updates on every `health_update` event
- ◆ Build `AlertPanel.jsx` — sidebar alert list. CRITICAL alerts pulse. Historical alerts scrollable
- ◆ Build `HistoryModal.jsx` — opens on 'View History'. Calls `GET /api/history`. Full sensor log table
- ◆ Design and apply consistent color scheme (Navy #0D1B3E, Blue #005B9E, Accent #00A8E8, Orange #F07D00, Green #007A3D)
- ◆ Prepare and rehearse the live demonstration flow — you are the one presenting the dashboard to judges

BUILD ORDER — WHAT TO BUILD FIRST

Follow this order strictly. Do not start Layer 5 (frontend) before Layer 1–2 (backend + simulation) is working. Build core, then connect, then polish.

Week	Priority	What to Build	Who
Week 1	🔑 Core Foundation	<code>Simulator.py</code> (MODE B) + <code>db.py</code> (PostgreSQL tables) + basic <code>app.py</code> + <code>fatigue_engine.py</code> . By end of Week 1: <code>python app.py</code> should generate sensor data, calculate health scores, and write to PostgreSQL. No frontend yet.	Prakash + Prince
Week 1–2	🔑 Real-Time Layer	<code>socketio_server.py</code> — Flask emitting <code>sensor_update</code> events every 2 sec. Simple test: open browser console and confirm Socket.IO events arriving. Then build <code>csv_reader.py</code> (MODE A) and mode-switch endpoint.	Prakash + Prince

Week 2	📁 ML Foundation	Piyush: preprocess_cmapss.py + generate_synthetic.py + train_model.py. Get aerotwin_model.pkl trained and working. Hand to Prakash for integration into Flask.	Piyush
Week 2	📁 Basic Frontend	Harnoor: Dashboard.jsx connecting to Socket.IO. Even just console.log-ing incoming events. Then HealthChart.jsx — first real visual. Get SOMETHING displaying on screen by end of Week 2.	Harnoor
Week 3	📁 Full Frontend	Harnoor: EngineModel.jsx (Three.js), RULPanel.jsx, AlertPanel.jsx, ModeToggle.jsx. All components built and receiving live data.	Harnoor
Week 3	📁 ML Integration	Prakash integrates aerotwin_model.pkl into Flask. ml_model.py running inference on every tick. RUL values appearing in Socket.IO events.	Prakash + Piyush
Week 4	📁 Anomaly + Alerts	alert_engine.py + anomaly z-score detector. Inject anomaly button working end-to-end: click → Flask spikes data → alert fires → dashboard shows CRITICAL.	Prakash + Harnoor
Week 4	📁 Polish	HistoryModal.jsx + /api/history endpoint. Feature importance widget. Dataset download documentation. README complete.	All
Week 5	📁 Testing + Demo Prep	Full end-to-end test both modes. Fix bugs. Harnoor rehearses demo script. Team rehearses Q&A answers.	All

Team FELONS | AeroTwin — Full & Final Project Report | InnoVent-27 | AI at the Edge | Confidential